

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №3-4
«Функциональные возможности языка Python»

Выполнил:
Студент группы ИУ5-31Б
Чичёв Иван

Проверил:
Гапанюк Ю. Е.

2025 г.

Листинг кода

cm_timer.py

```
from time import time
from contextlib import contextmanager

class cm_timer_1:

    def __enter__(self):
        self.start_time = time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("time:", time() - self.start_time)

@contextmanager
def cm_timer_2():
    start = time()
    yield
    print("time:", time() - start)
```

field.py

```
def field(items, *args):
    if not args:
        return
    if len(args) == 1:
        key = args[0]
        for d in items:
            val = d.get(key)
            if val is not None:
                yield val
    else:
        for d in items:
            new_dict = {}
            for key in args:
                val = d.get(key)
                if val is not None:
                    new_dict[key] = val

            if new_dict:
                yield new_dict
```

gen_random.py

```
import random

def gen_random(num_count, begin, end):
    for i in range(num_count):
        yield random.randint(begin, end)
```

print_result.py

```
def print_result(func):
    def wrapper(*args, **kwargs):

        print(func.__name__)

        result = func(*args, **kwargs)

        if isinstance(result, list):
            for i in result:
                print(i)
        elif isinstance(result, dict):
            for i in result:
                print(i, "=", result[i])
        else:
            print(result)

    return result
return wrapper
```

process_data.py

```
import json
import sys

from field import field
from print_result import print_result
from cm_timer import cm_timer_2
from gen_random import gen_random
from unique import Unique

path = sys.argv[1]

with open(path) as f:
    data = json.load(f)

@print_result
def f1(arg):
    return sorted(Unique((d['job-name'] for d in arg), ignore_case=True),
key=str.lower)
```

```

@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith("программист"), arg))

@print_result
def f3(arg):
    return list(map(lambda x: x + " с опытом Python", arg))

@print_result
def f4(arg):
    salaries = gen_random(len(arg), 100000, 200000)
    return [f"{name} с зарплатой {salary} рублей" for name, salary in zip(arg, salaries)]

if __name__ == '__main__':
    with cm_timer_2():
        f4(f3(f2(f1(data))))

```

sort.py

```

data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

result = sorted(data, key=abs, reverse=True)

result_lambda = sorted(data, key = lambda x: abs(x), reverse=True)

```

unique.py

```

class Unique(object):
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.used_values = set()
        self.ignore_case = kwargs.get("ignore_case", False)

    def __next__(self):
        while True:
            try:
                new_value = next(self.items)
            except StopIteration:
                raise StopIteration

            if self.ignore_case and isinstance(new_value, str):
                key = new_value.lower()
            else:
                key = new_value

            if key not in self.used_values:
                self.used_values.add(key)
                return new_value

```

```
def __iter__(self):  
    return self
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Программистр-разработчик информационных систем  
f3  
Программист с опытом Python  
Программист / Senior Developer с опытом Python  
Программист 1C с опытом Python  
Программист C# с опытом Python  
Программист C++ с опытом Python  
Программист C++/C#/Java с опытом Python  
Программист/ Junior Developer с опытом Python  
Программист/ технический специалист с опытом Python  
Программистр-разработчик информационных систем с опытом Python  
f4  
Программист с опытом Python с зарплатой 170155 рублей  
Программист / Senior Developer с опытом Python с зарплатой 167397 рублей  
Программист 1C с опытом Python с зарплатой 176372 рублей  
Программист C# с опытом Python с зарплатой 197415 рублей  
Программист C++ с опытом Python с зарплатой 193540 рублей  
Программист C++/C#/Java с опытом Python с зарплатой 143875 рублей  
Программист/ Junior Developer с опытом Python с зарплатой 101604 рублей  
Программист/ технический специалист с опытом Python с зарплатой 137378 рублей  
Программистр-разработчик информационных систем с опытом Python с зарплатой 199310 рублей  
time: 0.04355430603027344  
van@DESKTOP-260AC2V:~/git/Chichev-BKIT-labs-2025/lab_python_fp$
```